

ADMIRAL

NEVER SAY REFACTORING IS
IMPOSSIBLE AGAIN



ANDREAS KLEINBICHLER

```
{  
  first-name: "Andreas",  
  surname: "Kleinbichler",  
  jobtitle: "Engineering Manager",  
  company: "Admiral Technologies",  
  other: [  
    "iSAQB Trainer Foundation Level",  
    "Lecturer at FH Technikum Wien"  
  ],  
  lovesDevOps: true,  
  lovesTDD: true,  
  stack: [  
    "C#", "C/C++", "Angular", "SQL Server",  
    "MongoDB", "RabbitMq", "Docker", "K8s",  
    "AWS", "LINUX"  
  ]  
}
```

WM> WeAreDevelopers
World Congress Europe

8–10 July 2026 • Berlin

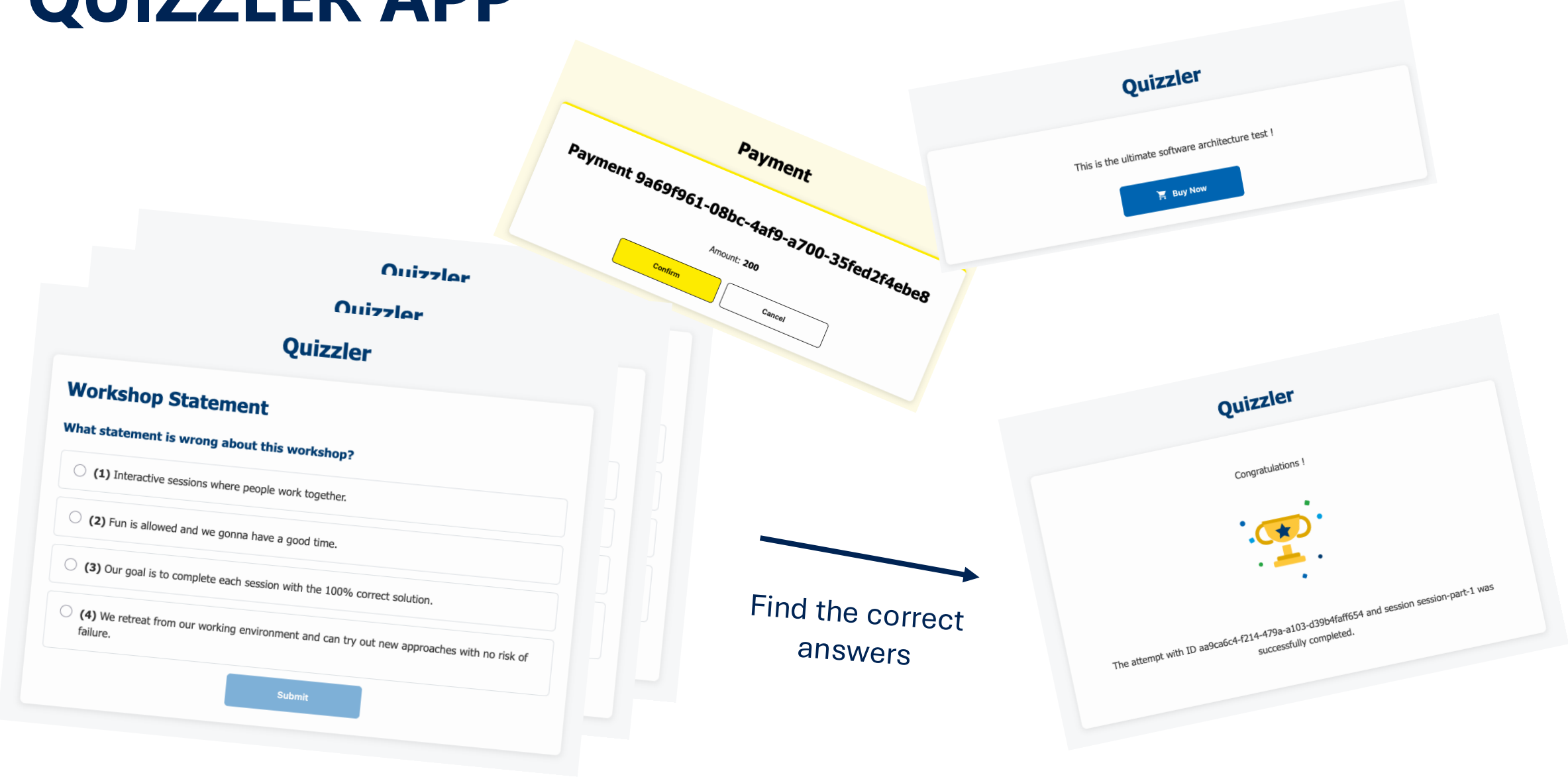


Andreas
Kleinbichler

Engineering Manager
at Admiral Technologies

Grow, connect, and lead in the age of AI worldcongress.dev

QUIZZLER APP



LOADING SPINNER ON THE MOVE

Why should we be concerned ?

The payment confirmation fails and it is not on the quizzler part of the system to realize it ?



We need downtime during new updates of the quizzler application.

A bug in the quizzler application breaks the payment.



We cannot change the endpoint of the quizzler api without involving the payment.



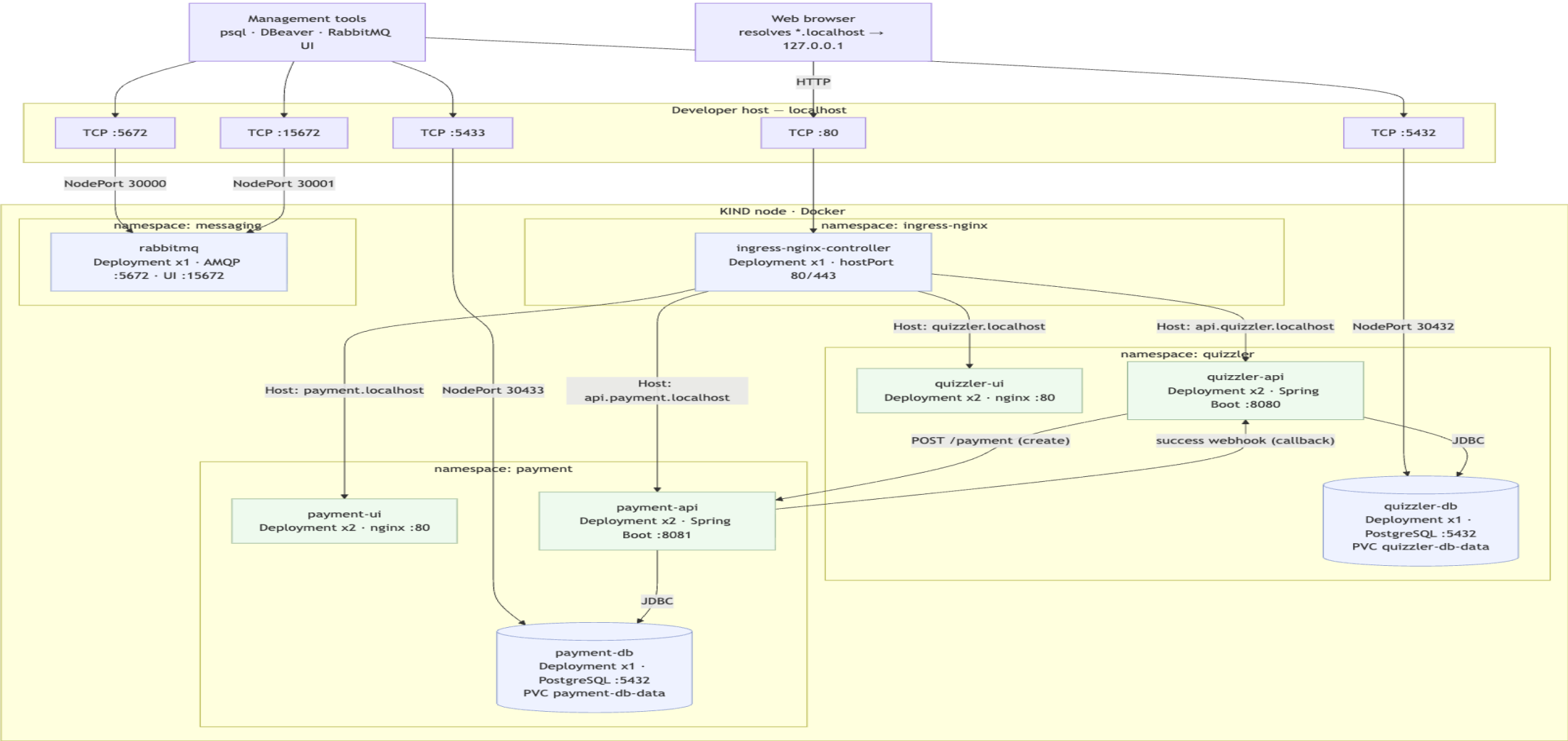
LOADING SPINNER ON THE MOVE

What could happen to us as a company if we don't change ?



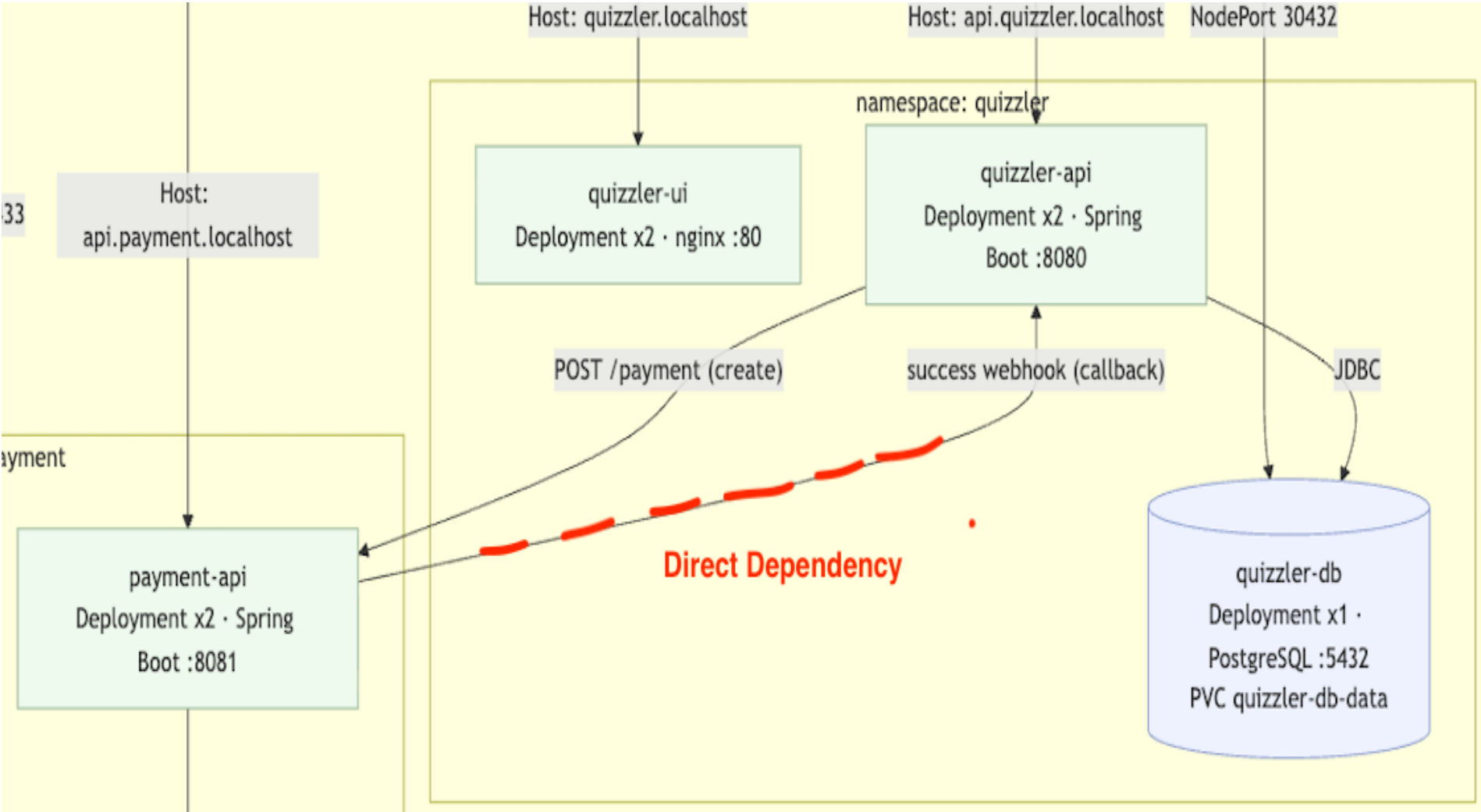
LOADING SPINNER ON THE MOVE

Architecture

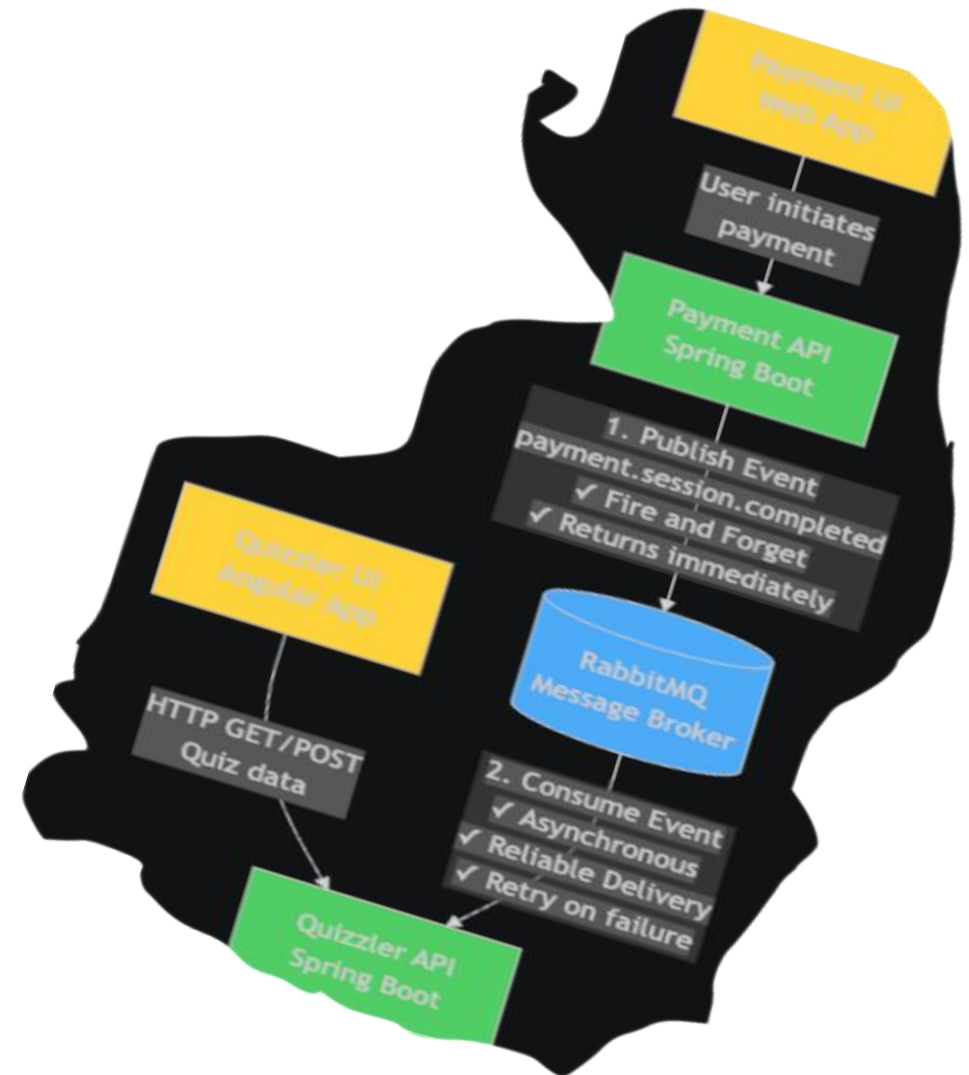
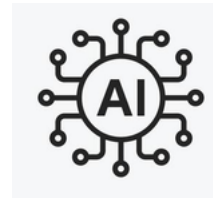
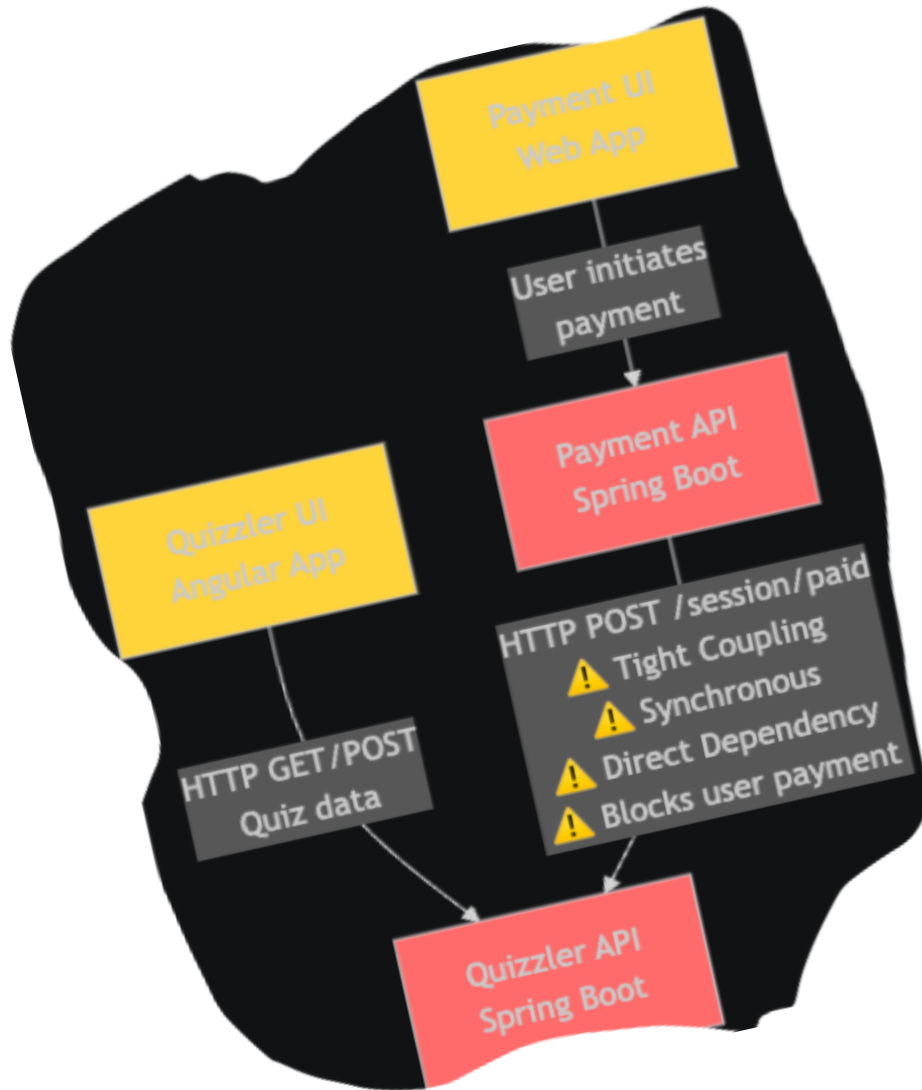


LOADING SPINNER ON THE MOVE

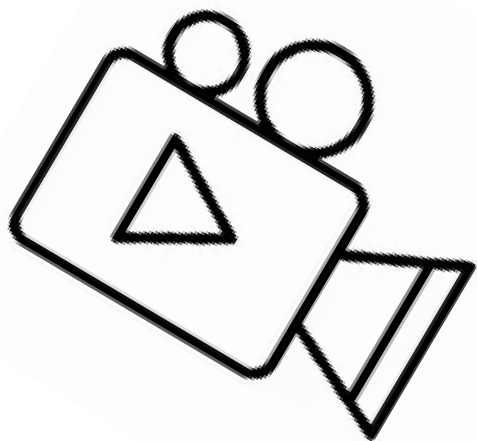
Problem Discovery



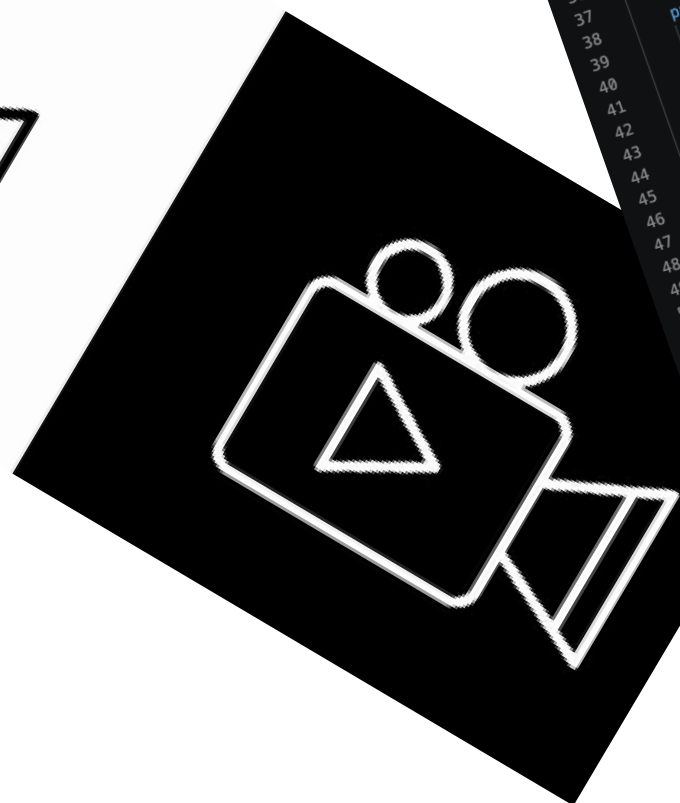
LOADING SPINNER ON THE MOVE / SOLUTION



LOADING SPINNER ON THE MOVE



How would you do this
change manually ?



```
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

@Transactional
public Payment confirmPayment(String paymentId) {
    Payment payment = paymentRepository.findById(paymentId)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
            "Payment " + paymentId + " not found"));
    if (paymentCancellationRepository.existsByPayment(payment)) {
        throw new ResponseStatusException(HttpStatus.CONFLICT,
            "Payment " + paymentId + " is already cancelled");
    }

    // A duplicate confirmation is rejected by the unique payment_id constraint at insert time,
    // so we let the database enforce it rather than issuing an extra existence query.
    PaymentConfirmation saved = paymentConfirmationRepository.saveAndFlush(
        new PaymentConfirmation(UUID.randomUUID().toString(), payment, Instant.now()));
    try {
        saved = paymentConfirmationRepository.saveAndFlush(
            new PaymentConfirmation(UUID.randomUUID().toString(), payment, Instant.now()));
    } catch (DataIntegrityViolationException ex) {
        throw new ResponseStatusException(HttpStatus.CONFLICT,
            "Payment " + paymentId + " is already confirmed", ex);
    }

    // Once the confirmation is persisted, notify the merchant by calling back the success-webhook
    // URL it supplied at payment creation. A webhook failure surfaces as 502 and rolls the
    // confirmation back, so the caller can safely retry the whole operation.
    confirmationWebhookClient.notifyConfirmation(payment.getWebhookSuccessUrl());
    return new PaymentConfirmationDto(saved.getPublicId(), payment.getPublicId(), saved.getCreatedAt());
}
```

How would you like the
AI doing it ?

LOADING SPINNER ON THE MOVE

Hands On

```
// Step 1: Take a deep breath
while (architecture.isBroken()) {
    refactor();
    test();
    deploy("without-downtime");
}
celebrate()
```

LOADING SPINNER ON THE MOVE

The fix

The loading spinner did move from the payment confirmation to the quiz-attempt-confirmation.

This means the payment is not blocked anymore, with all the consequences.

Problem solved.



LOADING SPINNER ON THE MOVE

Let's collection some findings ! Do you want to add something here ?

Refactoring Unknown Code

We CAN refactor code we don't know!

Don't let unfamiliarity stop you. With the right patterns and approaches, even legacy codebases can be safely refactored.

Arc42 CCC Section

Use the **Cross-Cutting Concepts (CCC)** section of Arc42 to enforce consistency during refactoring.

Document patterns, principles, and constraints upfront!

Strangler Pattern

Create a Skill / Strangler Skill

Gradually replace old systems by wrapping and delegating. Strangle the legacy code safely over time.

Infrastructure Matters!

Infrastructure must support your deployment strategy

Key considerations for zero-downtime refactoring:

- ▶ `Rolling Updates` strategy
- ▶ `SIGTERM` signal handling for graceful shutdown
- ▶ `Liveness` and `Readiness` probes
- ▶ Client **reconnect/retry** logic

Without proper infra, your perfect refactor will still cause downtime! 

GRAPH OF WISDOM

Graph of Wisdom Dashboard



With the quizzler app users can monitor their development of wisdom !!!

Wisdom Development Over Time

Session Dashboard

Session ID: session-003

Quiz Performance

Total Questions: 5
Answered: 5
Correct Answers: 4
Wrong Answers: 1

Accuracy
80%

Payment Information

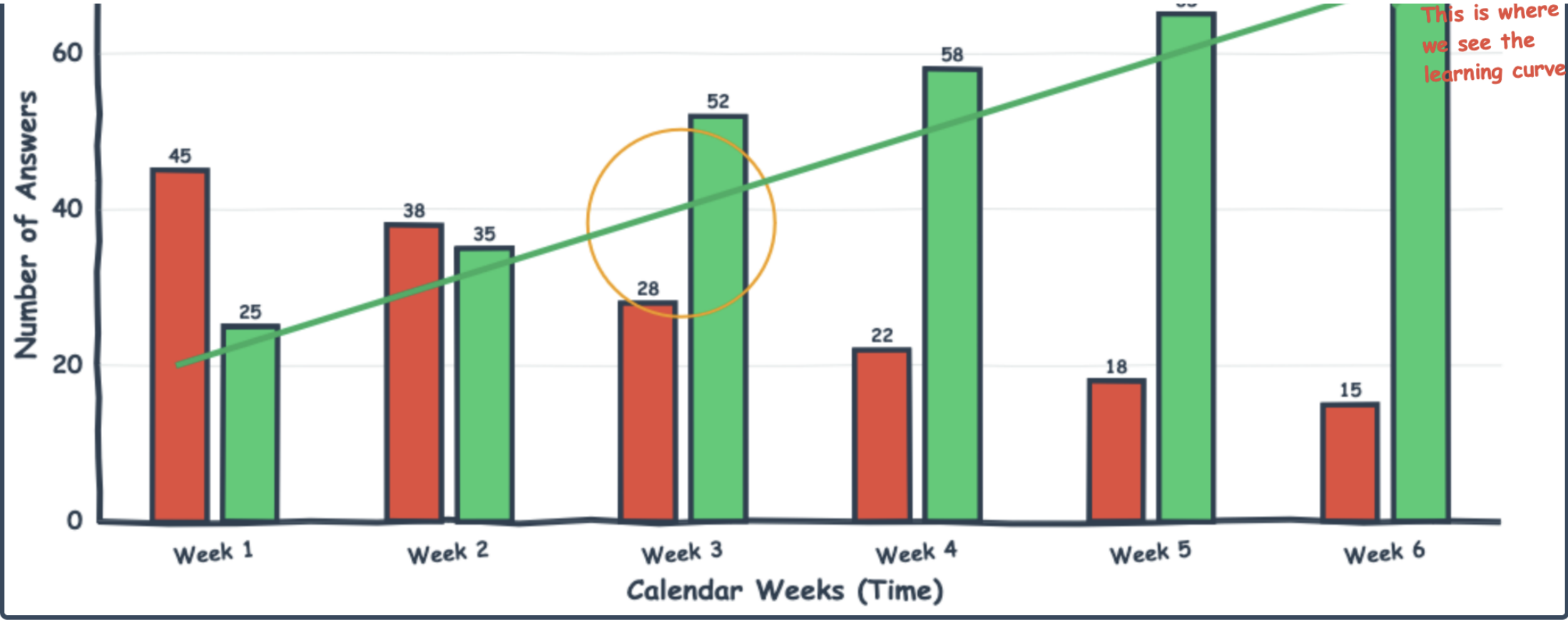
Number of Payments: 3
Total Payment Amount: \$1.50
Average per Payment: \$0.50

Graph of wisdom development coming soon



GRAPH OF WISDOM

The product vision



GRAPH OF WISDOM

Why should we be concerned ?



Our architecture is not flexible enough for the upcoming changes !

Our database design is not efficiently utilizing storage !



Deployments are complicated and risky due to database migrations.



High risk of regressions !



GRAPH OF WISDOM

What could happen to us when we don't change ?



GRAPH OF WISDOM

Architecture review

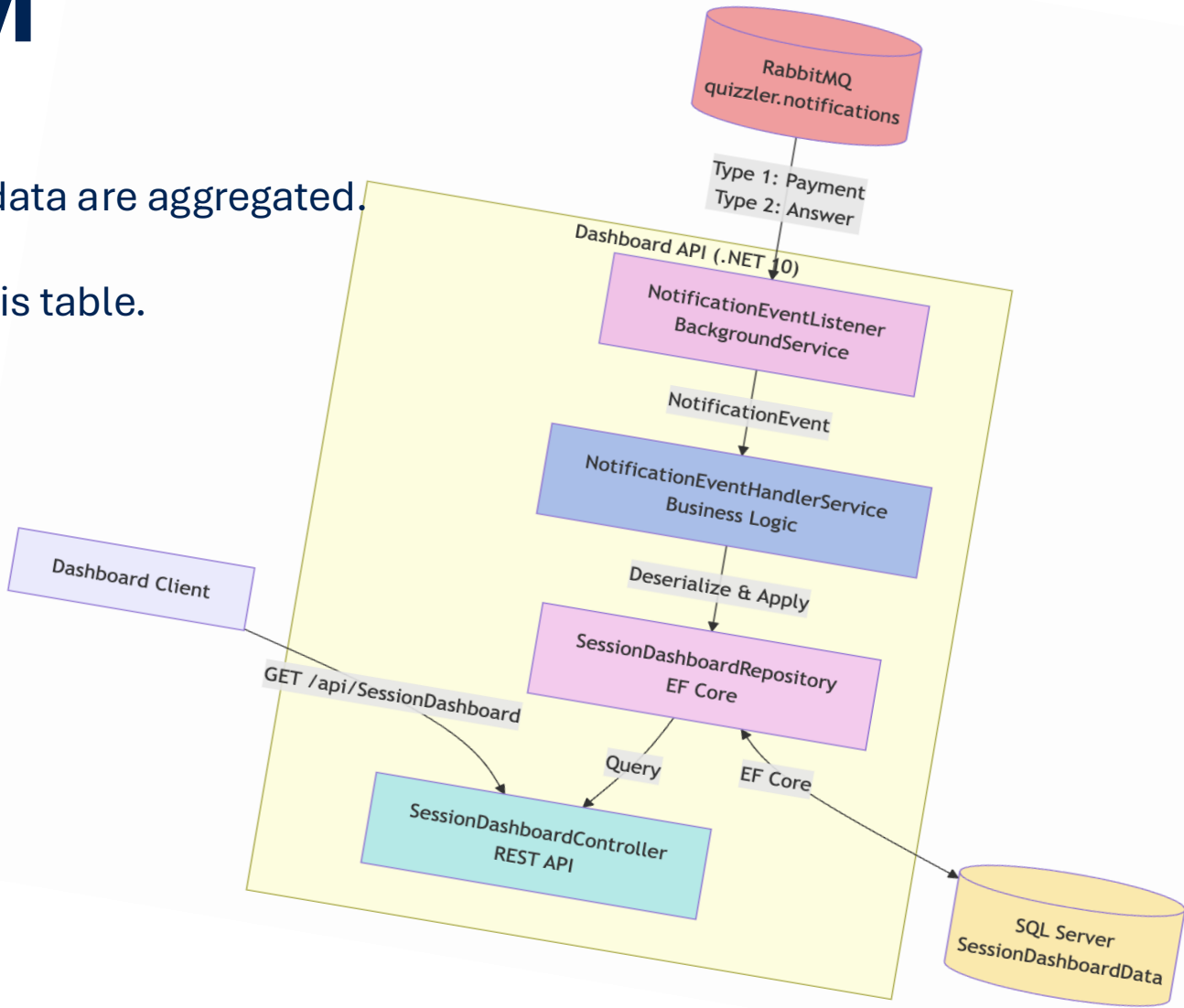
Through event sourcing the session dashboard data are aggregated.

Queries can be realized by just selecting from this table.

This is not supporting the graph of wisdom, as this bar diagram needs to be built on top of the detailed events !

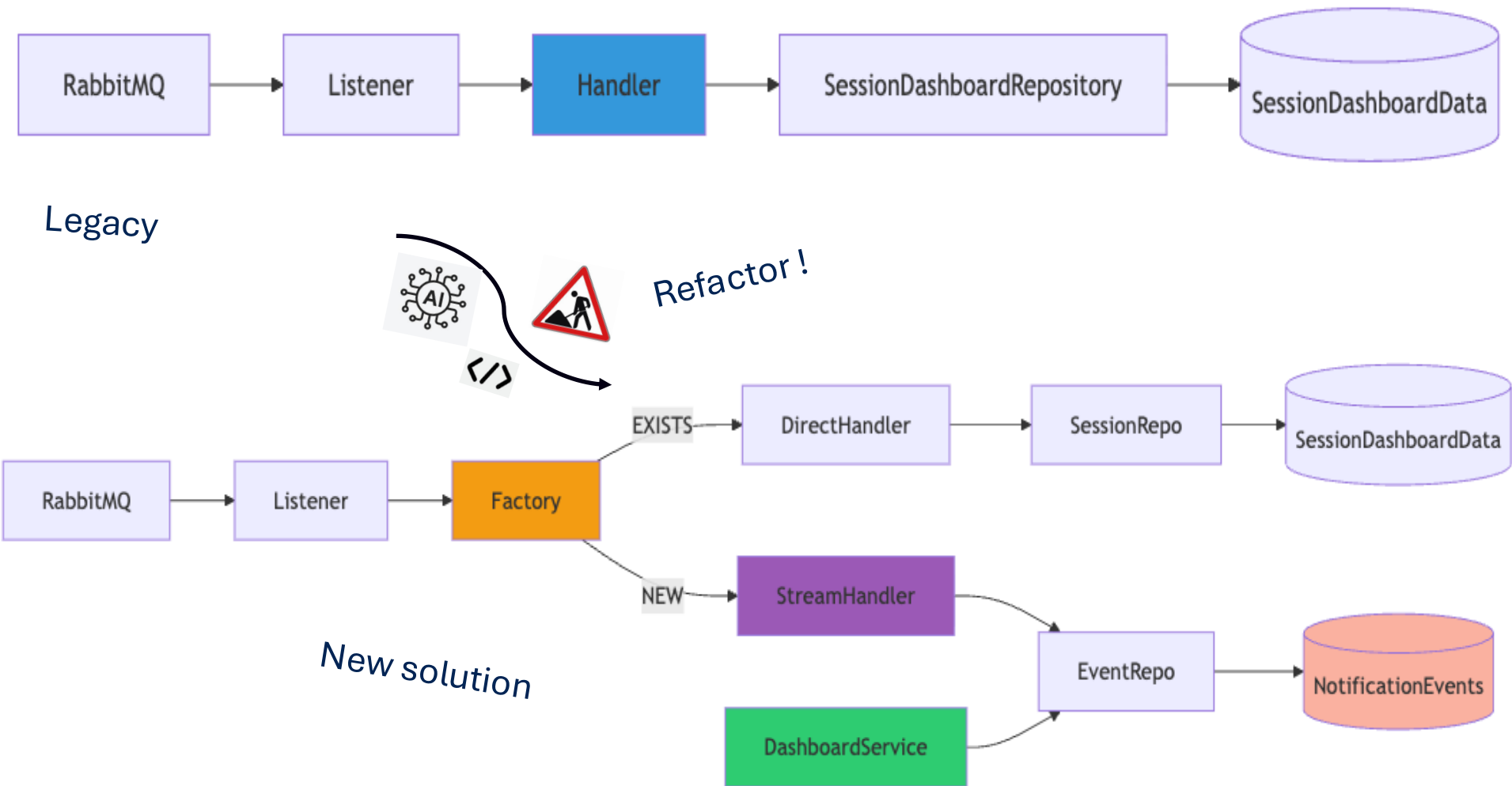
Database Schema: SessionDashboardData

Field	Type	Updated By
<u>Id</u>	int (PK)	Auto-increment
<u>DashboardId</u>	string	SessionId from events
<u>PaymentAmount</u>	int	Type 1 events (Purchase)
<u>NumberOfPayments</u>	int	Type 1 events (Purchase)
<u>WrongAnswers</u>	int	Type 2 events (Answer, IsCorrect=false)
<u>CorrectAnswers</u>	int	Type 2 events (Answer, IsCorrect=true)
<u>Questions</u>	int	Type 2 events (Answer)

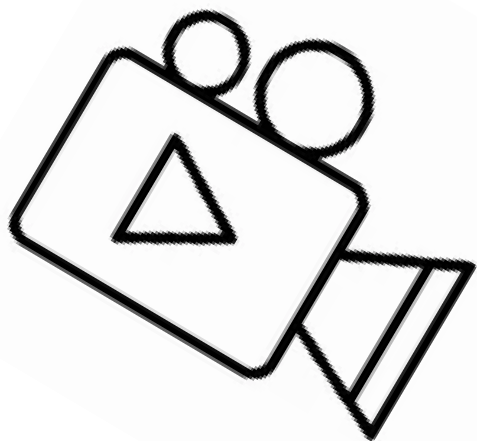


GRAPH OF WISDOM

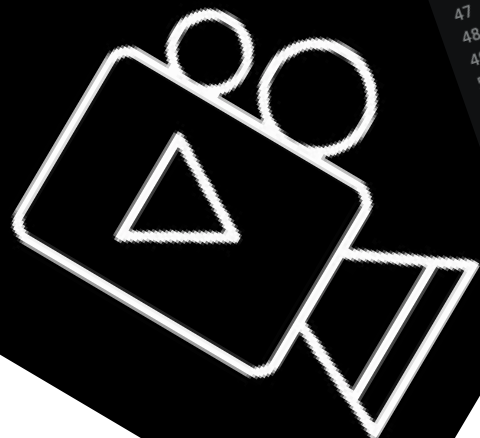
Think about a possible solution



GRAPH OF WISDOM



How would you do this
change manually ?



```
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

@Transactional
public PaymentConfirmationDto confirmPayment(String paymentId) {
    Payment payment = paymentRepository.findById(paymentId)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
            "Payment " + paymentId + " not found"));
    if (paymentCancellationRepository.existsByPayment(payment)) {
        throw new ResponseStatusException(HttpStatus.CONFLICT,
            "Payment " + paymentId + " is already cancelled");
    }
    // A duplicate confirmation is rejected by the unique payment_id constraint at insert time,
    // so we let the database enforce it rather than issuing an extra existence query.
    PaymentConfirmation saved;
    try {
        saved = paymentConfirmationRepository.saveAndFlush(
            new PaymentConfirmation(UUID.randomUUID().toString(), payment, Instant.now()));
    } catch (DataIntegrityViolationException ex) {
        throw new ResponseStatusException(HttpStatus.CONFLICT,
            "Payment " + paymentId + " is already confirmed", ex);
    }
    // Once the confirmation is persisted, notify the merchant by calling back the success-webhook
    // URL it supplied at payment creation. A webhook failure surfaces as 502 and rolls the
    // confirmation back, so the caller can safely retry the whole operation.
    confirmationWebhookClient.notifyConfirmation(payment.getWebhookSuccessUrl());
    return new PaymentConfirmationDto(saved.getPublicId(), payment.getPublicId(), saved.getCreatedAt());
}
```

How would you like the
AI doing it ?

GRAPH OF WISDOM

Hands On

```
// Step 1: Take a deep breath
while (architecture.isBroken()) {
    refactor();
    test();
    deploy("without-downtime");
}
celebrate()
```

GRAPH OF WISDOM

The improvement

New session dashboards are now created from a data stream.

This means that our architecture is now flexible.

System improved !



IMPROVE

GRAPH OF WISDOM

Findings ! Do you want to add some ?

✨ It's Not As Hard As Expected!

Fear of refactoring is often worse than the reality.

We thought moving to event sourcing would take weeks. The actual implementation? Just 4 small, incremental steps!

"We were paralyzed by imagined complexity. Turns out, one new table and a factory was all we needed."

🔄 Backward Compatibility is Key

The factory pattern saved us!

Old dashboards kept working with direct updates. New dashboards got event sourcing. No flag day, no migration hell.

"We didn't break existing users while building the future. That's the dream!"

🌀 Small Steps Beat Big Bang

Incremental refactoring wins every time!

Each step (1→2→3→4) was deployable and testable. No "big reveal" moment where everything breaks.

- ▶ Step 1: Add event table
- ▶ Step 2: Stream handler
- ▶ Step 3: Rehydration service
- ▶ Step 4: Factory routing

📦 Event Store = One Table

Event sourcing doesn't need complex infrastructure!

Just added `NotificationEvents` table. No Kafka, no event store framework. Simple SQL does the job.

- ▶ Id, SessionId, Type, Details, Timestamp
- ▶ That's it. Really!

GRAPH OF WISDOM

Findings ! Do you want to add some ?

Test Data Got 10x Easier

Event replay changed everything for testing!

Instead of crafting complex aggregate states, we just replay events. Want to test a dashboard with 100 answers? Just insert 100 events.

"Our test setup went from 50 lines to 10. And it's more realistic!"

Storage Concerns Were Overblown

Events are smaller than we thought!

We feared "event bloat". Reality: Events compress well, and most sessions are short-lived. Storage cost? Negligible.

- ▶ 200 bytes per event
- ▶ Average session: 10 events = 2KB
- ▶ 10,000 sessions = 20MB

Debugging Became Trivial

Event logs are debugging superpowers!

Bug report: "Dashboard shows wrong count"

Solution: Query events, replay, find the bad one.

"Before: 2 hours of guessing. After: 5 minutes of replaying events."

New Features Became Instant

The real payoff: Adding metrics is now trivial!

PM wants "average answer time"? Just add it to the DTO's `ApplyTo()` method, replay events. No schema migration. No downtime.

"What used to take a sprint now takes 20 minutes. This is the flexibility we needed!"

NEVER SAY REFACTORING IS IMPOSSIBLE AGAIN



THANK YOU !

THANK YOU !

THANK YOU !

THANK YOU !

THANK YOU !

THANK YOU !